Why Enforcement Requires Non-Participation: Structural Conditions for Coordination Integrity in Multi-Agent Systems

Navigational Cybernetics 2.5 — Coordination Integrity Architecture

Author: Maksim Barziankou (MxBv)
Contact: research@petronus.eu
Date: 2026-04-09

DOI: 10.17605/OSF.IO/9XZ8G
Axiomatic Core (NC2.5 v2.1): 10.17605/OSF.IO/NHTC5
Attribution: petronus.eu

Axiomatic Core (NC2.5 v2.1) — ■■■■■■■ ■■■■ ■ ■■■■■■■■■■■: DOI 10.17605/OSF.IO/NHTC5

1. The Problem: Coordination Fails Silently

Multi-agent systems comprising heterogeneous cognitive agents — large language models, autonomous executors, retrieval agents, human operators — operating over shared mutable state face a class of failures that are invisible from inside the task layer. These are not failures of individual agent competence. They are topological failures: structural breakdowns in how agents coordinate, not in what they compute.

Three failure modes define this class. First, coordination divergence: agents develop incompatible views of the shared state topology. Propagation records conflict, access control events contradict each other, trust-level assignments become inconsistent across nodes. No single agent is wrong in its local computation — but the system as a whole has lost coherence. Second, partial propagation: a state update reaches some memory stores but not others, producing asymmetric views that compound over subsequent operations. Third, cold-start amplification: when an agent recovers after absence, it faces a backlog of unapplied committed transitions whose count scales with operational history. The recovering agent must synchronize, but the divergence between its state and the system's state may already exceed any manageable bound.

These failures share a defining property: they are observable at the coordination layer without access to task-level semantic content. You do not need to understand what an agent was doing to detect that its propagation records conflict with another agent's, or that a trust assignment violated monotonicity. The failures are structural — and therefore, in principle, structurally enforceable.

In principle. The question is: by whom?

2. Why Monitoring From Within Does Not Work

The intuitive answer is supervision. Assign a more capable agent — or a designated monitor — to watch the coordination layer and intervene when invariants are violated. This is the approach taken by runtime verification systems, safety supervisors, and most enforcement agent frameworks in the literature.

The intuitive answer is wrong, and it is wrong for a structural reason, not a performance reason.

A monitor that participates in task-level computation develops internal state correlated with the task domain. Its reasoning is shaped by the same context windows, the same prompt structures, the same training distributions as the agents it monitors. When a coordination invariant is violated in a way that correlates with the task domain — and in systems of any complexity, most violations do — the monitor's detection capacity is compromised by the same biases that produced the violation. The monitor does not fail because it is insufficiently intelligent. It fails because its intelligence is of the same kind as the agents it watches.

This is not a problem that better models solve. A more capable model from the same architectural family shares the same systematic capability gaps. If a particular class of numerical precision errors is invisible to GPT-family architectures, a larger GPT will not see them either. The blind spot is architectural, not parametric.

Shared architecture produces shared blind spots. Shared blind spots produce correlated monitoring failure. Correlated monitoring failure means that the very violations the monitor exists to detect are the ones it is least likely to catch.

No amount of privilege, access, or computational budget resolves this. The problem is not what the monitor can see — it is what the monitor cannot see because of what it is.

3. The Non-Participation Principle

The structural answer is not a better participant but a non-participant. *Nemo iudex in causa sua* — no one may judge their own cause. This legal principle, older than any computational framework, encodes an architectural insight: impartiality is not a property of judgment quality but of structural position.

An enforcement agent whose authority derives from non-participation must satisfy a precise structural condition: it holds no task-level credentials, receives no task-level requests, does not access the task corpus, and does not generate task-level outputs. This is not a design preference. It is the precondition for enforcement authority. If the agent participates in task computation, its internal state becomes a function of task content. Any enforcement decision it makes for coordination conditions whose violation signals correlate with that content is then subject to the same state dependencies as the task agents themselves. The enforcement is no longer independent. It is no longer enforcement — it is self-regulation by another name.

Non-participation must be enforced at the dispatch layer, not by policy. The coordination system's routing infrastructure must exclude the enforcement agent from task-level request queues. The agent must lack credentials for task-level data stores. Its input must be restricted to coordination-level event records — structured metadata with scalar or enumerated payloads. Free-form text, prompt bodies, generated content, raw memory payloads: all excluded by schema. The enforcement agent operates on topology, not on semantics.

This restriction is not a limitation that weakens the enforcer. It is the structural condition that makes enforcement possible. An enforcement agent that can see task content is an enforcement agent that can be corrupted by task content. Content blindness is the source of authority, not its absence.

4. Enforcement as Non-Causal Structural Constraint

This architecture establishes a fundamental separation between enforcement and intervention. The enforcement agent does not cause correct behavior. It does not provide gradient signal, reward, correction, or guidance to task-level agents. It establishes a boundary condition on the coordination topology: if a coordination invariant is violated, propagation on the affected edge is suspended. That is all.

The suspension is not an action in the task domain. It is a predicate on the coordination topology: this edge does not propagate until the condition is restored. Task-level agents do not receive information about *why* the edge was suspended in terms of their task semantics. They observe a structural state — halt or not halt — and nothing more.

This is the runtime expression of a deeper architectural principle: enforcement operates as a non-causal structural constraint, not as a causal intervention. It does not tell agents what to do. It defines what cannot propagate. The distinction matters because causal intervention entangles the enforcer with the task domain — it must understand the violation to correct it, and understanding the violation requires task-level access that voids the non-participation condition. Non-causal constraint avoids this entirely. The enforcer does not understand the violation. It detects a structural condition (a binary predicate on coordination metadata) and applies a structural consequence (propagation suspension on a typed edge).

In the language of Navigational Cybernetics 2.5: admissibility is a non-causal structural predicate that constrains realization without providing gradient signal, optimization objective, or actionable geometry. It does not say "do X." It says "Y cannot be realized." The enforcement agent is the runtime instantiation of this principle — binary evaluation (pass/fail per condition per event), no gradient, no proximity signal, no reward shaping. The conditions of possibility for coordination integrity are established structurally, not caused dynamically. Kant's distinction applies precisely: what is at stake is not the efficient cause of coordinated behavior, but the conditions under which coordination is possible at all.

5. Architectural Heterogeneity as Correlated Failure Prevention

Non-participation addresses the correlation between enforcer and task domain. A second structural condition addresses the correlation between enforcer and monitored agents at the architectural level.

If the enforcement agent shares the same model family, the same training corpus, and the same reasoning architecture as the agents it monitors, it shares their systematic capability gaps. A class of coordination violations that is undetectable by one architecture — because detection requires capabilities absent in that architecture class — is equally undetectable by the enforcer. The system has no independent detection capacity for that violation class. Every agent in the system, including the enforcer, is blind in the same way.

This is not a performance gap. It is a structural correlation. Addressing it requires provenance differentiation: the enforcement agent's architectural provenance — at minimum its model family and reasoning architecture class — must differ from that of every active task-level agent on at least one dimension.

This requirement is not diversity for robustness in the usual sense. It is not ensemble averaging or N-version programming where multiple implementations vote on a result. The enforcement agent does not vote. It enforces. And it can only enforce independently if its detection capacity is not correlated with the detection failures of the agents it monitors. Provenance differentiation is a structural hedge against correlated blind spots — not a guarantee of independence, but a necessary condition for non-trivial independent enforcement.

6. The Coupled Architecture: Five Co-Required Properties

The architecture described here comprises five properties. Individually, each has precedent: access control restricts scope, diversity reduces correlation, monitors detect violations, circuit breakers halt propagation, schema constraints limit input. The novelty is not in any single property. It is in the co-required conjunction and in the categorical consequences of breach.

The five properties are: non-participation enforced at the dispatch layer; architectural heterogeneity verified against provenance attributes; continuous condition monitoring over coordination event streams with binary satisfaction signals; propagation halt authority limited to typed coordination edges without corrective capability; and input scope restriction to schema-constrained coordination metadata excluding all task-level semantic content.

The conjunction is necessary because the failure modes are architecturally coupled. Removing any one property reintroduces a qualitatively distinct failure class that the remaining four cannot prevent:

Without non-participation, the enforcer develops state correlation with the task domain. Its enforcement decisions become subject to the same biases as the agents it monitors. The failure mode is structural

loss of independence.

Without architectural heterogeneity, the enforcer shares systematic capability gaps with monitored agents. Violations detectable only through capabilities absent in the shared architecture class go undetected by everyone. The failure mode is correlated blind spots.

Without continuous condition monitoring, the architecture reduces to a static gatekeeper that can be triggered externally but cannot autonomously detect violations from the coordination event stream. The failure mode is reactive-only enforcement.

Without halt authority, the architecture reduces to a passive monitor. It detects violations and emits alerts but cannot prevent propagation of violating state transitions. The failure mode is detection without containment.

Without input scope restriction, the enforcer receives task-level signals that are unnecessary for coordination invariant evaluation but that introduce channels through which task-level biases can influence enforcement decisions. The failure mode is scope contamination — the independence established by non-participation is undermined through the input channel.

Prior approaches address these failure modes individually: access control for scope, diversity for correlation, circuit breakers for halting, monitors for detection. The present architecture recognizes that these failure modes are coupled. Eliminating any single one while leaving the others in place does not produce enforcement independence. The conjunction is the architectural primitive, not the individual components.

7. Categorical Invalidity vs Gradual Degradation

Conventional safety systems degrade gracefully. A monitor that misses a signal is a worse monitor, not an invalid one. A circuit breaker that trips too slowly is a slower circuit breaker, not a structurally compromised one. The implicit assumption is that partial capability is better than none.

For coordination enforcement under the architecture described here, this assumption is wrong. If the enforcement agent breaches non-participation — if it accesses task content, even once — its internal state is no longer independent of the task domain. Every subsequent enforcement decision is potentially correlated with task state. The system cannot distinguish between an enforcement action that reflects independent structural evaluation and one that reflects task-domain bias. The trust is gone, and it is gone categorically, not gradually.

The same holds for architectural heterogeneity. If the enforcer's provenance attributes converge with those of a monitored agent on all dimensions, correlated blind spots are no longer a risk — they are a structural property of the configuration. Enforcement actions taken under this condition carry no independence guarantee.

The architectural response is binary authority. The enforcement agent either satisfies all five co-required properties and has full halt authority, or it fails any one and has none. There is no intermediate state of partial enforcement. This is not a harsh policy choice — it is a structural consequence. Partial authority in the presence of structural compromise is indistinguishable from compromised authority. The only safe response is suspension.

8. Implications for Multi-Agent Coordination Design

The enforcement agent described in this architecture is not a cognitive agent. It does not think, plan, optimize, or learn. It does not generate outputs, interact with users, or participate in task execution. It is a structural artifact — a dedicated process that receives a bounded stream of coordination metadata, evaluates binary conditions, and suspends propagation on typed edges when conditions are violated.

This represents a distinct architectural class. It is not reinforcement learning — there is no reward signal, no policy optimization, no exploration. It is not planning — there is no goal state, no search, no trajectory generation. It is not safe-RL — there is no constraint integration into a learning objective. It is not

constrained optimization — there is no objective function. It is not general monitoring — the five co-required properties with categorical invalidity on breach are not features of any prior monitoring architecture.

The class is defined by a structural principle: enforcement authority derives from separation, not from capability. The enforcer is not a smarter agent. It is a structurally isolated one. Its power comes from what it cannot do — access task content, share architectural provenance with monitored agents, modify agent state, remediate violations — not from what it can.

For multi-agent systems that operate over shared mutable state across time horizons exceeding the continuous presence of any single agent — systems where agents arrive and depart, models are updated, context windows are truncated — static constraints are necessary but insufficient. Coordination invariants must be actively enforced. And the enforcer must be structurally separated from the domain it governs, architecturally differentiated from the agents it monitors, and categorically invalidated when these conditions fail.

The alternative is self-regulation. And self-regulation, as every coordination failure at scale has demonstrated, is not regulation at all.

This work is part of Navigational Cybernetics 2.5 (NC2.5), a formal theory of long-horizon adaptive systems. Published under CC BY-NC-ND 4.0.

■■■■■■■ ■■■■ ■ ■■■■■■■■■■■ *— NC2.5 v2.1: DOI 10.17605/OSF.IO/NHTC5*

PETRONUS™ — petronus.eu — research@petronus.eu

MxBv, Pozna■ 2026.

Content Integrity Verification

■ 11-Layer Structural Protection (PETRONUS PPCP Protocol)

Publication: 2026-04-09T11:48:01Z

Integrity Generated: 2026-04-09T11:48:01.715Z

Layer	Method	Value
1	SHA-256 Content Hash	`363540d61cb9ada493fb85aa309f0bbdbdbea9a9b7476d8a8a7b298a41d9f3c3`
2	Merkle Root (9 sections)	`8f3fda6ccf8a8406baa91e2beb05f73bbc3325fa89e7e0d831837d7fd23d6e63`
3	HMAC-SHA256	`6b83acb1c473aba671fdffa646aa2f559d2b888acf59c8c821e71015f1478ce7`
4	Publication Timestamp	2026-04-09T11:48:01Z
5	Metadata Fingerprint	`28ffefee74ed36238994698bdeedb31dab0e4e3da48b15553c59b5e826d1e34d`
6	Structural Fingerprint	`28ffefee74ed36238994698bdeedb31dab0e4e3da48b15553c59b5e826d1e34d`
7	Steganographic Watermark	Embedded (ZWC)
8	Image Integrity (SHA-256)	`1627aa68eccae1368bb0432ef0d4c001ff9ee651d8d329b799464e33a41e8729`

9	Git Commit Signature	[populated-at-commit]
10	Copyright / DMCA	© 2025–2026 Maksim Barziankou. All rights reserved. CC BY-NC-ND 4.0
11	Audit Log Entry	ENP-001

Structural Profile:

- 2397 words · 119 lines · 9 sections

- 9 headings

Verification: Any modification to content, structure, or metadata invalidates layers 1–3 and 5–8 simultaneously. Structural fingerprint detects section reordering, insertion, or deletion. Steganographic watermark survives copy-paste but not re-encoding. Image hash validates cover integrity independently.

Full hashes available on request for independent verification.

Maksim Barziankou (MxBv) · PETRONUS · research@petronus.eu
petronus.eu | DOI: 10.17605/OSF.IO/9XZ8G
Axiomatic Core (NC2.5 v2.1): 10.17605/OSF.IO/NHTC5